

Orb Weaver / Website ORB Manufacturing Build — Detailed Handoff Summary

Date: August 19, 2026

1. Current objective

The immediate objective is to complete the commercial Website ORB manufacturing system.

The working chain is:

Orb Weaver
↓
scan / weave website
↓
compile verified site intelligence
↓
manufacture customer payload
↓
clone Website_Orb_Final golden runtime
↓
inject customer payload
↓
initialize customer storage / learning
↓
place/configure Website ORB in Dock Station package
↓
validate complete package
↓
deploy

The first authoritative real-world manufacturing target remains **TrueMark**, followed by another site to prove that the manufacturing process is repeatable and that site knowledge does not leak between customer ORBs.

The immediate job is **not** to improve the Campaign site and **not** to build the Desktop ORB.

2. The four things we are actually building

A. Orb Weaver — the manufacturer

Orb Weaver is responsible for transforming a website into the information required by a Website ORB.

Its job is to:

- discover the website;
- fetch/render pages;
- inspect public and authorized owner/admin areas;
- understand routes;
- extract site/business facts;
- extract products;
- extract services;
- extract inventory;
- extract prices;
- extract SKUs;
- extract variants;
- extract availability;
- extract specifications;
- extract business identity/contact information;
- extract policies;
- build semantic relationships;
- build Site World;
- build Pointer Plot Map;
- build pointer correspondence;
- build catalog;
- build A Priori knowledge;
- collect provenance/evidence;
- present material requiring owner verification;
- compile the customer payload;
- clone the golden ORB;
- initialize clean learning state;
- validate;
- package.

Orb Weaver is the factory.

B. Website_Orb_Final — the golden ORB

`Spruked/Website_Orb_Final` is intended to be the **site-neutral master Website ORB that Orb Weaver clones**.

It is not supposed to contain a customer's identity or Orb Weaver's identity.

Every manufactured customer ORB begins with the same reusable runtime and receives different data.

Current repo structure already includes substantial pieces:

```
Website_Orb_Final/  
├── backend/  
├── frontend/  
├── compiled_orb/  
├── Orb_Vault_System/  
├── architecture/  
├── docs/  
├── tests/  
├── tools/  
├── vendor/  
└── PACKAGE_MANIFEST.json
```

The repo already contains the Vault-first/TPC architecture and much of the pointer/runtime foundation.

But it is **not yet a completed golden clone**.

C. Dock Station — owner control/deployment package

The Dock Station is the owner-facing control/configuration environment.

The ORB belongs **inside the Dock Station package architecture**, rather than the Dock Station being conceptually subordinate to the ORB.

The repository already reflects that intended structure in the older manufacturing template:

```
manufacturing/  
└─ templates/  
    └─ dock_station_master/  
        └─ dock-station/  
            └─ app/  
                └─ orb/  
                    └─ template/
```

However, that current ORB template is only the old placeholder:

```
assets/  
deployment/  
runtime/
```

It is **not Website_Orb_Final**.

The functional **Orb_Assistant/user_dock_station/** implementation contains considerably more real Dock Station capability and will have to be reconciled with the manufacturing Dock Station rather than maintained forever as two unrelated implementations.

D. Customer Website ORB

This is the final manufactured deployment.

It receives:

- generic immutable/release-managed engine;
- customer identity;
- customer ORB name;
- customer site payload;
- customer A Priori;
- customer catalog;
- customer Site World;
- customer pointers;
- clean customer A Posteriori;
- customer memory/storage;
- customer voice/configuration;
- owner settings;
- package manifest.

No customer should inherit another customer's learned state.

No customer should inherit Orb Weaver's own Site World.

3. Hard scope boundary

For this work:

IN SCOPE

- Orb Weaver
- scanning/weaving
- catalog extraction
- payload compilation
- Site World
- pointers
- A Priori
- A Posteriori
- persistent Website ORB storage
- voice
- Faster Whisper STT
- Kokoro TTS
- local Qwen
- provider adapters for larger models
- articulation/professional inculcation
- Dock Station
- Website_Orb_Final
- packaging
- verification
- deployment

OUT OF SCOPE

- Desktop ORB
- Desktop fleet orchestration
- R-drive/Desktop computer automation
- Campaign site rebuild

The standard Website ORB can retain a **future-compatible connection boundary**, but we are not building the system on the other end of it now.

4. Orb Weaver is the donor implementation

A major decision from this review is that we should **stop recreating Website ORB functionality that already works inside Orb Weaver**.

That was part of the purpose of building Orb Weaver's ORB so extensively.

The Orb Weaver documentation actually calls its current voice implementation the **gold-master implementation for copying the repaired Website ORB voice runtime into future ORBs**.

Therefore:

Orb Weaver live Website ORB



reusable pieces



Website_Orb_Final

We copy the proven generic behavior, then remove Orb-Weaver-specific data/configuration.

5. Website_Orb_Final currently has major missing pieces

The current repo is not yet a complete voice-first deployable Website ORB.

Its current backend requirements are only:

- FastAPI
- Uvicorn
- Pydantic

There is no complete self-contained STT/TTS/local-model runtime represented by those dependencies.

The current frontend has states called **listening** and **speaking**, but it is essentially a text interface. It does not yet implement Orb Weaver's proven microphone recording/end-of-speech voice lifecycle.

So the golden clone needs a donor-copy pass.

6. Voice runtime that should come from Orb Weaver

The proven Orb Weaver lifecycle is:

one click
→ microphone capture
→ automatic speech detection
→ automatic end-of-speech
→ server STT
→ answer resolution
→ backend TTS/cache
→ one WAV
→ playback
→ idle

This should be brought into [Website_Orb_Final](#).

The known working Orb Weaver implementation already provides:

- [navigator.mediaDevices.getUserMedia\(\)](#);
- [MediaRecorder](#);
- RMS/silence monitoring;
- automatic stop after speech ends;
- hard maximum recording duration;
- cancellation;
- interruption;
- ref-based single-flight lock;
- [AbortController](#);
- latency filler audio;
- one request per turn;
- one returned WAV;
- cleanup on failure;
- cleanup on playback completion.

It specifically avoids:

- browser [SpeechRecognition](#);
- browser [speechSynthesis](#) as final voice;
- multiple submission requests;
- repeated TTS retry storms.

That behavior should be copied rather than recreated.

7. Faster Whisper

The Website ORB should use the same proven server STT model as Orb Weaver.

Browser:

audio/webm

Backend:

POST Faster Whisper STT

Expected result:

```
{  
  "text": "visitor's spoken request"  
}
```

The Website ORB therefore needs the reusable STT adapter and error handling from Orb Weaver.

8. Kokoro TTS

Kokoro belongs in the deployable Website ORB architecture.

Orb Weaver already contains the reusable Kokoro FastAPI implementation.

It supports:

POST /speak

POST /v1/audio/speech

and can use CUDA when available.

The Website ORB needs:

- Kokoro runtime/install requirements;
- selected Website ORB voice;
- TTS service configuration;
- TTS cache;
- single-flight protection;
- pre-generated common lines;
- dynamic response generation.

The cache is especially important.

Orb Weaver's testing showed the difference between a cold generated line and a cached response can be enormous.

Common responses should therefore be pre-generated/cached:

- greeting;
- identity response;
- listening acknowledgment;
- latency filler;
- temporary failure;
- completion;
- confirmation;
- goodbye.

Dynamic customer-specific answers remain generated.

9. Local LLM

The Website ORB needs a small local model lane.

Current preferred architecture uses the local OpenAI-compatible interface so the Website ORB is not tightly coupled to a particular inference server.

Existing Dock configuration already defines a Qwen 2.5 1.5B local-primary model through an OpenAI-compatible endpoint.

The golden package therefore needs the **runtime/launcher/configuration required to run or connect to that local model**, not merely a hardcoded URL that exists only on the development computer.

The model itself is an escalation layer, not the first thing called.

10. Larger LLM provider adapters

The Website ORB/Dock package also needs the optional adapter layer for larger external models.

The existing provider router already supports:

- local OpenAI-compatible;
- OpenAI;
- Anthropic;
- Google;
- custom OpenAI-compatible providers.

It includes primary/fallback routing and API-specific request translation.

That should be reused.

We do **not** need separate bespoke ORB implementations for GPT, Claude, Gemini, etc.

They all sit behind one provider interface.

Credentials must remain outside browser JavaScript.

11. Articulation / professional inculcation

This is another missing core component that needs to be incorporated correctly.

Current:

Orb_Assistant/user_dock_station/articulation-contract.js

is mostly a loader.

It reads:

artifacts/inculcation.md

and passes that doctrine to the provider.

The important content is the actual professional inculcation/doctrine.

It includes:

- Core Four reasoning baseline;
- Website ORB professional identity;
- evidence discipline;
- role responsibilities;
- tool truth;
- authority separation;
- pointer/MORB doctrine;
- operational-state awareness;
- articulation constraints;
- final doctrine checksum.

That material belongs in the generic Website ORB engine/delivery architecture, not as an accidental dependency on an Orb Weaver-relative file.

12. Articulation should not destroy the fast path

The existence of an articulation model does **not** mean every response should go through an LLM.

The intended latency architecture remains:

Known deterministic answer



direct response



TTS

Example:

“How much is the Alpine tent?”

should preferably be:

STT

→ catalog.db lookup

→ exact current price

→ response template

→ Kokoro

No LLM required.

The generative model is used when interpretation/composition is genuinely needed.

13. The deterministic resolution hierarchy

The intended hot-path order is approximately:

1. control command
2. exact catalog lookup
3. A Priori
4. verified A Posteriori
5. precompiled Site World / route knowledge
6. TPC
7. local Qwen
8. optional larger external model
9. articulation/doctrine validation
10. Kokoro

The goal is:

do not pay probabilistic inference latency for something already known.

14. Current Vault integration

`Website_Orb_Final` already has a meaningful Vault implementation.

The current answer engine attempts:

A Priori

→ A Posteriori

→ TPC fallback

before continuing down heavier paths.

The Vault coordinator explicitly defines:

1. A Priori catalog;
2. A Priori ontology/QA;
3. verified A Posteriori;
4. TPC/LLM escalation.

That basic approach is correct.

15. A Priori

A Priori is **settled site/business truth**.

Examples:

- business identity;
- products;
- services;
- prices;
- policies;
- hours;
- contact information;
- known FAQs;
- approved terminology;
- relationships;
- verified route meanings;
- verified business facts.

Current `Website_Orb_Final` expects:

catalog.json
ontology.json
qa.json
policies.json

and contains loader logic for these structures.

We are now refining the catalog lane toward SQLite because product/inventory/pricing data is relational and should support exact fast lookup.

16. A Posteriori

A Posteriori is learned operational experience.

It is not merely conversation history.

It should capture things such as:

- successful resolution patterns;
- recurring visitor intents;
- verified phrasing correspondences;
- repeated useful relationships;
- successful support outcomes;
- learned site-use patterns.

Lifecycle:

candidate
→ verification
→ reinforcement
→ promotion
→ merge
→ weaken
→ retire

A new customer receives a **clean** A Posteriori.

A customer's learned state must never be copied into another customer's ORB.

Age alone is not a reason to destroy useful learned knowledge.

17. Institutional knowledge over years

The Website ORB should be capable of becoming more valuable over a long business lifetime.

A 10- or 20-year business may want to retain a great deal of institutional learning.

The system should distinguish between:

Active operational facts

Example:

Product X costs \$199

If discontinued, this should leave the active catalog.

Historical/institutional knowledge

Example:

Customers frequently compare this type of product to Y.

or:

This seasonal purchasing pattern has repeated for several years.

That may remain valuable.

The substrate therefore should not behave like an aggressive cache that simply deletes old intelligence.

18. Customer/visitor memory is separate from A Posteriori

This distinction is important.

A Posteriori

= what the ORB learns about performing its job

Customer memory

= what the ORB is allowed to remember about a particular visitor/customer

Those should be governed separately.

Customer memory requires appropriate consent/permissions and should be individually removable without destroying general ORB learning.

19. Persistent Website ORB substrate/storage

The installed Website ORB needs persistent storage beyond its Day-1 manufactured payload.

The ORB will accumulate:

- A Posteriori;
- customer memory;
- long-term knowledge;
- workflow state;
- verified outcomes;
- indexes;
- caches;
- logs;
- manifests;
- runtime state;
- updated site intelligence;
- backups.

The recommended design is **not** to allocate a literal 5 GB disk image immediately.

Instead:

create storage root

→ allow dynamic growth

→ apply a quota

Default recommendation:

5 GB soft quota

with owner expansion available later.

For example:

warning at 70%

automatic maintenance at 80%

hard protection around 95%

owner can increase allocation

A high-volume or long-lived business can later increase this to 10 GB, 25 GB, etc.

20. One persistent storage authority

The deployed ORB should converge toward a single authoritative persistent root.

Conceptually:

```
vault_system/  
├── posteriori/  
├── customer_memory/  
├── short_term_memory/  
├── long_term_memory/  
├── workflow_state/  
├── verified_outcomes/  
├── databases/  
├── indexes/  
├── runtime_state/  
├── persistent_cache/  
│   └── tts/  
├── audit/  
├── manifests/  
└── backups/
```

Reusable **Vault code** can live in the immutable engine.

Persistent **Vault data** goes into this storage root.

That is cleaner than allowing multiple unrelated persistent Vault directories.

21. Self-maintenance and pruning

Storage management should understand data classes.

Safe/reasonable automated cleanup

- obsolete TTS cache;
- temporary files;
- superseded indexes;
- stale runtime logs;
- duplicate embeddings;
- redundant candidate records.

Carefully governed maintenance

- old customer memory;
- A Posteriori;
- historical business knowledge;
- verification evidence;
- owner-approved records.

The ORB should know whether something is:

- active;
 - historical;
 - regenerable;
 - compressible;
 - mergeable;
 - owner-controlled;
 - non-deletable.
-

22. The Site Payload Injection Bundle

The customer-specific data must remain separate from the reusable engine.

The original concept you supplied was:

```
orb_client_package/  
├── core_engine/  
└── payload/  
    ├── catalog.db  
    ├── apriori.json  
    ├── pointers.json  
    └── site_config.json
```

That basic principle is correct.

We refined it so complex A Priori components do not get flattened into one giant JSON document.

Recommended direction:

```
payload/  
├── site_config.json  
├── catalog.db  
├── apriori/  
│   ├── ontology.json  
│   ├── qa.json  
│   └── policies.json  
├── site_world.json  
├── pointers.json  
├── pointer_correspondence.json  
├── runtime_language.json  
├── tool_cache.json  
└── payload_manifest.json
```

The initial A Posteriori state can be initialized into persistent `vault_system/` during manufacture/first boot.

23. **catalog.db** — inventory lane

SQLite is appropriate for exact structured business inventory.

Core product table:

```
products
-----
product_id
name
sku
current_price
sale_price
currency
availability
source_url
```

Additional fields can include:

- variant;
- category;
- specifications;
- pointer target;
- source evidence;
- crawl version;
- verification;
- content hash.

Services:

```
services
-----
service_id
name
package_name
price_tier
description
source_url
```

Business facts can use another structured table.

This lane gives the ORB extremely fast deterministic responses.

24. `site_config.json`

This becomes the identity/configuration root for the manufactured site.

For example:

```
{
  "client_name": "Alpine Ridge Outfitters",
  "orb_name": "Scout",
  "base_url": "https://example.com",
  "default_voice": "weaver",
  "active_lanes": [
    "catalog",
    "apriori",
    "posteriori"
  ],
  "created_at": "..."
}
```

25. The owner must be able to name the ORB

This was added explicitly in the latest discussion.

The ORB's public name should **not be hardcoded in the golden runtime**.

The customer should be able to name it:

- while Orb Weaver manufactures it;
- during Dock Station setup;
- or later through owner settings.

That name should flow into:

- greeting;
- identity;
- UI;
- spoken responses;
- Dock Station;
- manifest;
- relevant metadata.

Changing the ORB's name should not require modifying engine code.

26. Pointer guidance

The manufactured payload must preserve semantic-to-physical correspondence.

Example:

```
{  
  "target_id": "target_598ed",  
  "intent_association": "CREDIT_APPLICATION",  
  "element_selector": "#financing-apply-btn",  
  "route": "/financing",  
  "stability_score": 0.98  
}
```

But our canonical version should also preserve:

- source URL;
- entity ID;
- verified state;
- allowed actions;
- content hash;
- semantic label;
- intent aliases.

Knowledge does not automatically authorize action.

Pointer/runtime verification remains separate.

27. Canonical Full Scan Evidence contract

Before payload compilation, Orb Weaver needs one canonical evidence contract.

Each compiler should operate from the same normalized evidence.

Conceptually:

evidence_id
evidence_type
source_url
route
selector
pointer_target_id
payload
confidence
verified
content_hash
crawl_version
build_version

Evidence types include:

- product;
- service;
- plan;
- fee;
- business fact;
- contact;
- FAQ;
- policy;
- navigation;
- pointer target.

The compiler transforms evidence.

It must **not invent facts**.

28. Verification

Not everything detected by a crawl becomes authoritative A Priori automatically.

Owner verification needs to support:

- approve;
- reject;
- correct/edit;
- inspect source;
- compare conflicts;
- review evidence;
- final build signoff.

Result:

verification_manifest.json

Policies in particular should not become authoritative merely because an LLM generated a plausible summary.

29. Dock Station current situation

There are presently two architectural generations visible in Orb Weaver.

Older manufacturing Dock Station template

`manufacturing/templates/dock_station_master/`

This has the right general package hierarchy, but the ORB template inside it is mostly empty.

Functional `user_dock_station`

`Orb_Assistant/user_dock_station/`

This contains substantial real functionality:

- Electron UI;
- owner profiles;
- control plane;
- articulation contract;
- provider router;
- credentials;
- Kokoro;
- model gateway;
- packaging;
- MCP-related code.

Its location under `Orb_Assistant/` is likely historical organization.

The final work should reconcile the useful functionality into the actual deployable Dock Station architecture rather than maintaining two incompatible Dock Stations.

30. Desktop MCP code currently inside Website_Orb_Final

This is one area that should be corrected.

Current:

backend/dock_adapter/dockstation_adapter.py

does little more than wrap `ORBDesktopMCPClient`.

And:

orb_desktop_mcp.py

contains desktop-machine capabilities including:

- mouse;
- click;
- drag;
- screen capture;
- OCR;
- clipboard;
- app opening;
- window listing;
- browser automation.

That is not required for a normal customer Website ORB.

The standard Website ORB should retain only whatever narrow Dock Station configuration/control interface it genuinely needs.

The Desktop/R-drive automation machinery should not become a mandatory dependency.

31. Website_Orb_Final must be de-site-specific

The current golden candidate still contains Orb Weaver-specific identity.

Examples include the package manifest referencing:

target_domain: orbweaver.spruked.com

and frontend language that says:

“I know where I am on Orb Weaver.”

Those must become payload/config-driven.

A genuine golden clone should know **nothing about its eventual customer until Orb Weaver injects the payload.**

32. Manufacturing sequence

The target complete build process is:

1. Orb Weaver crawl/weave
2. raw evidence collection
3. normalize Full Scan Evidence
4. validate evidence
5. extract catalog
6. compile A Priori
7. compile Site World
8. compile runtime language
9. compile pointer map
10. compile pointer correspondence
11. compile tool cache
12. owner verification
13. write verification manifest
14. clone Website_Orb_Final
15. inject payload
16. configure ORB name/voice/owner settings
17. initialize empty A Posteriori
18. initialize persistent vault_system
19. configure storage quota
20. package inside Dock Station delivery structure
21. package validation
22. runtime tests
23. export/deploy
24. live-site verification

No manual customer-specific code edits should be required.

33. Transactional boundary

Orb Weaver may inspect and understand transactional surfaces.

It may identify:

- checkout;
- signup;
- financing;
- NFT minting;
- crypto interfaces;
- forms;
- wallet controls;
- admin interfaces.

But the scanner should not cause state-changing actions merely to understand them.

Do not:

- purchase;
- mint;
- sign a wallet transaction;
- transfer crypto;
- submit checkout;
- create destructive admin changes;
- submit arbitrary forms for discovery.

Observation and semantic understanding are different from execution.

34. TrueMark remains the first serious manufacturing test

TrueMark is useful specifically because it is not a perfect sanitized test site.

It exposes:

- commerce;
- transactional flows;
- NFT/minting surfaces;
- crypto;
- admin surfaces;
- pointer complexity;
- rendered-DOM challenges.

The goal is not to hand-build a great TrueMark ORB.

The proof is:

Orb Weaver manufactures it correctly.

Then the process gets repeated on another site to prove it wasn't a one-off.

35. Repository audit — Website_Orb_Final

The GitHub repository currently has **only main**.

Latest remote commits are:

cb6087e Update README for vault-first Website ORB architecture
036d6d3 Initial Website ORB final template baseline

Both were created August 19, 2026.

That means there are no additional remote Website_Orb_Final feature branches currently waiting to be reconciled.

The GitHub side of that repo is straightforward.

What I **cannot see from here** is whether:

C:\Users\bryan\Desktop\website_orb_final

contains:

- uncommitted changes;
- local-only commits;
- ignored files that matter;
- an older **main**;
- an ahead/behind state.

That final local check must happen through the machine terminal/Codex.

36. Repository audit — Orb Weaver

This one is **not currently consolidated**.

Remote `main` is:

cae4f113

Snapshot current Orb Weaver before full readiness evaluation

dated August 10, 2026.

But important development exists outside `main`.

`fix/website-orb-vault-manufacturing-contracts`

This is **10 commits ahead of `main`**.

It contains:

- `full_scan_evidence.v1.json`
- `catalog.v1.json`
- `ontology.v1.json`
- `qa.v1.json`
- `policies.v1.json`
- `pointer_correspondence.v1.json`
- `verification_manifest.v1.json`
- `compile_vaults.py`
- compiler tests

This work is not on `main`.

`orb-integrity-fix3-guidance`

This is **19 commits ahead of `main`**.

It contains substantial Website ORB/runtime work including:

- large `backend/main.py` changes;
- `AutonomousOrb.tsx`;
- pointer recovery;
- crawler changes;
- tests;

- frontend API work;
- ORB loading/runtime work;
- pointer end-to-end proof.

local-orb-weaver-snapshot-2026-08-10

This is **8 commits ahead of main** and includes much of the repaired working runtime:

- `AutonomousOrb.tsx`;
- `backend/main.py`;
- Kokoro changes;
- pointer recovery;
- crawler dynamic rendering;
- account/crawl pages;
- API changes;
- tests.

feature/tesseract-lidar-weaves

This is **7 unique commits ahead and 1 behind main**.

Unique work includes:

- Tesseract weave;
- LiDAR guidance mapping;
- tests.

fix/truemark-rendered-pointer-scan

This is **7 commits ahead / 5 behind**.

Includes:

- rendered DOM crawler;
- workflow;
- tests;
- crawl assembly status.

fix/customer-map-crawl-seeds

3 ahead / 5 behind.

Includes the crawler seed contract.

restore/lidar-2d-mapping-package

10 ahead / 5 behind.

Contains the restored LiDAR 2D mapping/coordinate-cache package.

feature/orb-inference-three-engine-gateway

25 ahead / 90 behind.

This is an old divergent line but contains potentially reusable inference work:

- inference gateway;
- providers;
- contracts;
- telemetry;
- llama.cpp deployment scripts;
- other engine experiments;
- benchmarks.

This should be **reviewed for useful reusable pieces**, not blindly merged wholesale.

fix/preflight-pointer-ping-light

8 ahead / 94 behind.

Old divergent work involving:

- local pointer fallbacks;
- movement controller;
- web actuator.

Again: inspect unique useful logic, do not blindly merge an obsolete tree.

recovered-marketplace

2 ahead / 163 behind.

Old branch containing marketplace recovery work.

Inspect whether that functionality already exists in the newer tree before deciding it has unique material.

Fully superseded/behind-only branches

These currently contain no commits ahead of `main`:

- `vault-consolidation`
- `fix/ga4-project-settings`
- `desktop-orb-now-surface`

Therefore they do not contain remote commits that must be imported into the current build.

The Desktop branch remains out of scope anyway.

37. Important conclusion from the branch audit

We cannot simply tell Codex:

“Pull main and begin.”

That would be wrong.

`main` does **not** contain all current work.

Nor should Codex blindly merge every historical branch.

The correct operation is:

fetch everything

- preserve local work
- identify unique remote commits
- consolidate active Website ORB / scanner / manufacturing work
- resolve superseded duplicates
- run tests
- push one known integration baseline
- make local and remote identical
- record starting SHAs
- THEN begin the new build

That is how we get the clean slate you want.

38. Clean-slate rule before Codex writes new code

Codex should make **zero new feature changes** until both repositories pass the preflight.

For each repo:

```
git fetch --all --prune --tags
git status
git branch -vv
git log --graph --decorate --oneline --all
```

Any uncommitted/local-only work gets preserved first.

Do **not**:

```
git reset --hard
git clean -fd
force push
delete branches
```

until the provenance of all work is understood.

Once consolidation is complete, the active branch must satisfy:

```
git status --porcelain
```

→ **empty**

and:

```
git rev-list --left-right --count HEAD...@{upstream}
```

→

```
0 0
```

That means:

- nothing uncommitted;
- nothing untracked that should be versioned;
- nothing waiting to push;
- nothing waiting to pull.

Then record:

git rev-parse HEAD

for both repos.

Those two SHAs become the **Codex START baseline**.

When Codex finishes the manufacturing pass, record the ending SHAs.

That gives us exactly the before/after record you're asking for.

39. Recommended starting-state record

Before Codex starts coding, we should end up with a small record like:

ORB WEAVER / WEBSITE ORB MANUFACTURING START BASELINE

Date: 2026-08-19

Orb Weaver

repo: Spruked/Orb_Weaver

branch: <consolidated branch>

HEAD: <sha>

working tree: clean

upstream divergence: 0 / 0

Website ORB Final

repo: Spruked/Website_Orb_Final

branch: main or build branch

HEAD: cb6087e... or newer reconciled SHA

working tree: clean

upstream divergence: 0 / 0

Scope:

- Orb Weaver
- Website_Orb_Final
- Dock Station
- Website ORB manufacturing
- payload architecture
- voice
- Vault
- models
- provider adapters
- packaging

Excluded:

- Desktop ORB
- Campaign site

Then every change Codex makes can be compared against that starting point.

40. The immediate build order after repository reconciliation

Once the repos are genuinely synchronized, the coding sequence should be:

1. Make Website_Orb_Final genuinely site-neutral.
2. Donor-copy the proven Orb Weaver Website ORB voice runtime.
3. Add Faster Whisper STT.
4. Add Kokoro runtime + cache + voice assets.
5. Add local Qwen runtime/provider configuration.
6. Add optional GPT/Claude/Gemini/custom provider routing.
7. Integrate complete articulation/professional inculcation.
8. Remove/quarantine unnecessary Desktop MCP dependencies.
9. Normalize the Website ORB Vault into the final single-persistence model.
10. Implement catalog.db.
11. Finalize the payload loader contract.
12. Add ORB naming/identity configuration.
13. Implement persistent storage initialization and quota handling.
14. Update Orb Weaver's Full Scan Evidence contract.
15. Finish catalog extraction.
16. Finish A Priori compilation.
17. Finish pointer correspondence compilation.
18. Finish owner verification manifest.
19. Make Orb Weaver clone Website_Orb_Final mechanically.

20. Inject payload mechanically.
21. Initialize clean A Posteriori mechanically.
22. Assemble Dock Station + Website ORB delivery package.
23. Validate the entire generated package.
24. Run fixture/dry-run tests.
25. Manufacture TrueMark's ORB.
26. Deploy and test TrueMark.
27. Repeat on second site.

That is the complete working picture we have established.

The **very next action before Codex writes anything is repository consolidation and clean-baseline verification**, because the remote audit proves Orb Weaver currently has meaningful work distributed across several branches, while Website_Orb_Final's remote state is much simpler. Once the local terminal confirms and reconciles that, we can give Codex one comprehensive build prompt against known starting SHAs.